Experiment # 07

Implementation of File I/O Redirection System Calls in Linux- II

Objective

In this lab we will learn about system call techniques in UNIX.

Software Tools

- Ubuntu 16.04
  - GCC Complier

Theory

System Call:

System calls are commands that are executed by the operating system. "System calls are the only way to access kernel facilities such as file system, multitasking mechanisms and the inter-process communication primitives. File Structure related System Calls:

- dup()
- dup2()

dup()

The dup() system call creates a copy of a file descriptor.

- It uses the lowest-numbered unused descriptor for the new descriptor.
- If the copy is successfully created, then the original and copy file descriptors may be used interchangeably.
- They both refer to the same open file description and thus share file offset and file status flags.

Syntax:
int dup(int oldfd);

where **oldfd** is old file descriptor whose copy is to be created.

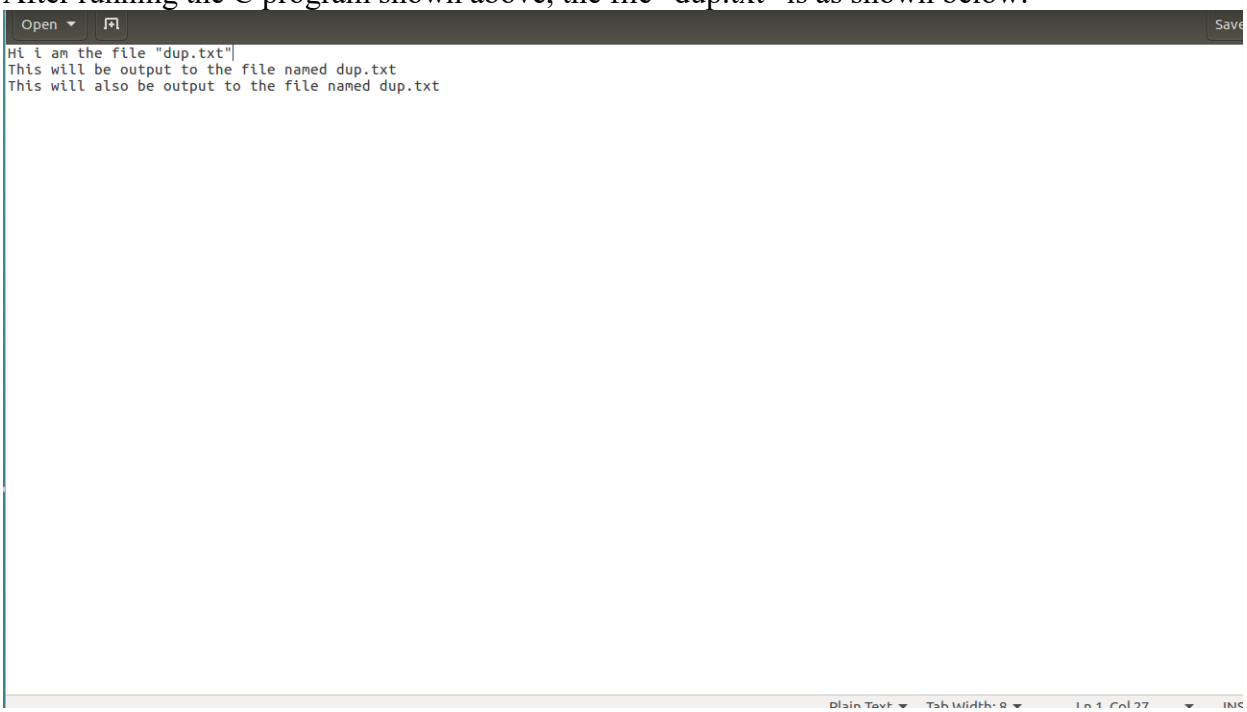
```
// CPP program to illustrate dup()
#include<stdio.h>
#include <unistd.h>
#include <fcntl.h>
```

```
int main()
{
   // open() returns a file descriptor file_desc to a
   // the file "dup.txt" here"

   int file_desc = open("dup.txt", O_WRONLY | O_APPEND);

   if(file_desc < 0)
      printf("Error opening the file\n");

   // dup() will create the copy of file_desc as the copy_desc
   // then both can be used interchangeably.

   int copy_desc = dup(file_desc);

   // write() will write the given string into the file
   // referred by the file descriptors

   write(copy_desc,"This will be output to the file named dup.txt\n", 46);

   write(file_desc,"This will also be output to the file named dup.txt\n", 51);

   return 0;
}
```

Explanation: The open() returns a file descriptor file_desc to the file named “dup.txt”. file_desc can be used to do some file operation with file “dup.txt”. After using the dup() system call, a copy of file_desc is created copy_desc. This copy can also be used to do some file operation with the same file “dup.txt”. After two write operations one with file_desc and another with copy_desc, same file is edited i.e. “dup.txt”.

Before running the code, Let The file “dup.txt” before the write operation be as shown below:

After running the C program shown above, the file “dup.txt” is as shown below:



dup2()

The dup2() system call is similar to dup() but the basic difference between them is that instead of using the lowest-numbered unused file descriptor, it uses the descriptor number specified by the user.

Syntax:

int dup2(int oldfd, int newfd);

where **oldfd** is old file descriptor and **newfd** is new file descriptor which is used by dup2() to create a copy.

Important points:

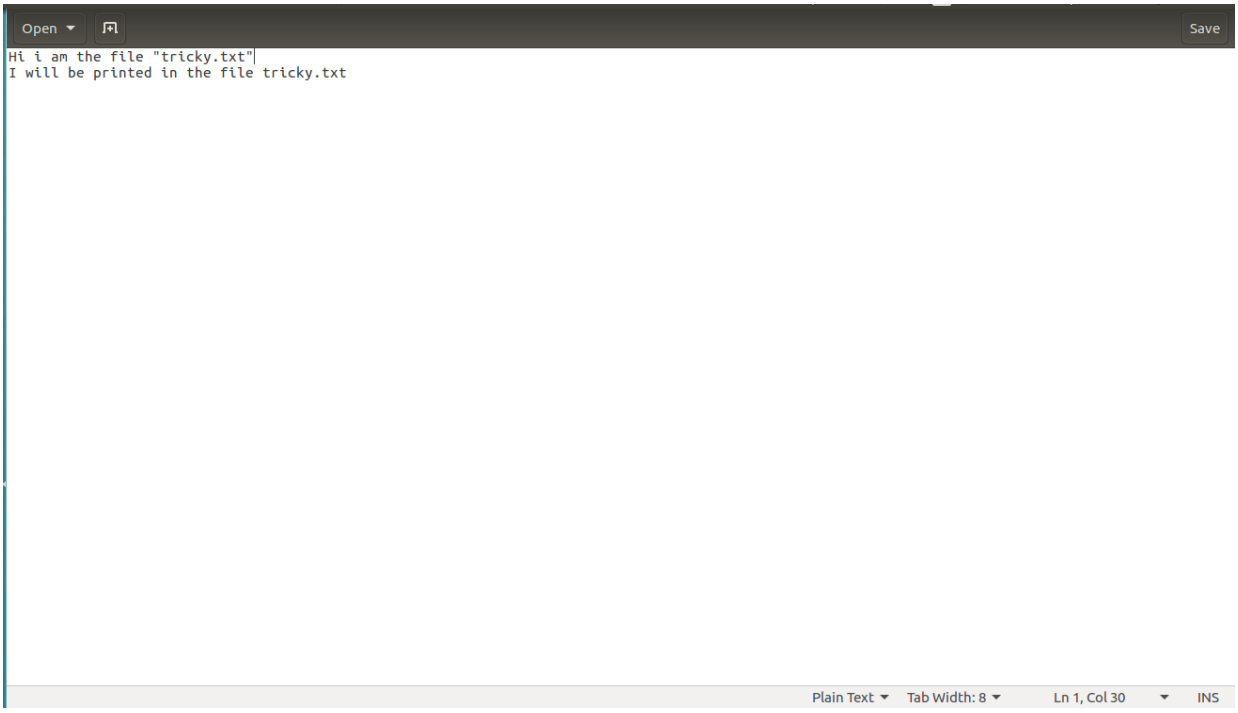
- Include the header file unistd.h for using dup() and dup2() system call.
- If the descriptor newfd was previously open, it is silently closed before being reused.
- If oldfd is not a valid file descriptor, then the call fails, and newfd is not closed.
- If oldfd is a valid file descriptor, and newfd has the same value as oldfd, then dup2() does nothing, and returns newfd.

A tricky use of dup2() system call: As in dup2(), in place of newfd any file descriptor can be put. Below is a C implementation in which the file descriptor of Standard output (stdout) is used. This will lead all the printf() statements to be written in the file referred by the old file descriptor.

```
// CPP program to illustrate dup2()
#include<stdlib.h>
#include<unistd.h>
#include<stdio.h>
#include<fcntl.h>

int main()
{
   int file_desc = open("tricky.txt",O_WRONLY | O_APPEND);

   // here the newfd is the file descriptor of stdout (i.e. 1)
   dup2(file_desc, 1) ;

   // All the printf statements will be written in the file
   // "tricky.txt"
   printf("I will be printed in the file tricky.txt\n");

return 0;
}
```

This can be seen in the figure shown below:
Let The file “tricky.txt” before the dup2() operation be as shown below:

After running the C program shown above, the file “tricky.txt” is as shown below:


Open
Save
Hi i am the file "tricky.txt"
I will be printed in the file tricky.txt
Plain Text
Tab Width: 8
Ln 1, Col 30
INS

Lab Task:

1. **Execute all the examples given in the manual and analyze the output.**
2. **Open a new file for reading and writing. Using duplicate descriptor dup(), Read the predefined number from the text file and calculate its factorial. Now write the output number to the same file as:**
   Factorial is: output_number.
3. **Write a C program which copy the data from file "SP.txt" to file "LAB.txt" using duplicate descriptor dup2() and display the output using open, close, read and write system calls.**

Lab Performance Evaluation:
Read and practice the manual. Performance is evaluated at the end of lab based on successfully running and understanding of examples, tasks and viva using defined rubrics.

Lab Report Evaluation:
Submit the Lab Report by writing all the examples and tasks which must include the following:

1. Brief description about how its work (3-5 lines)
2. The code
3. The results (Screenshot)

Important note: Lab Report must be according to the Lab Report Format available on Google Classroom and must be submitted on time. Two similar reports will get zero marks. So, don't try to copy your fellow reports. Lab report must be submitted in group of three maximum.